smard-forecast

Wahrscheinlichkeits-Prognose für Solar, Wind an Land und Wind auf See bis 6 Tage voraus, als neues Tool für smard-mcp

Projektbericht: Ziel, Daten, Technik, Ablauf, Betrieb und Entstehung, Schritt für Schritt

Stand: 2. November 2026
Repository: github.com/akquisio-dev/smard-forecast
Zielprojekt: github.com/akquisio-dev/smard-mcp (v1.3.0)
Status: Phase 0 abgeschlossen, Phase 1 läuft (Download und Archiv)

Inhalt

1 Kurzfassung . . . 4
Die wichtigsten Befunde bisher . . . 4
2 Ausgangslage und Ziel . . . 5
smard-mcp heute . . . 5
Was fehlt . . . 5
Ziel . . . 5
Was ausdrücklich nicht dazugehört (Version 1) . . . 5
3 Grundbegriffe . . . 6
Wettermodell, Lauf und Vorlaufzeit . . . 6
Quantile: p10, p50, p90 . . . 6
Auslastung . . . 7
LightGBM . . . 7
4 Das Vorlaufzeit-Problem . . . 8
Zwei Archive bei Open-Meteo . . . 8
Was „Tag N“ genau bedeutet (geprüft) . . . 9
5 Datenquellen . . . 10
Open-Meteo . . . 10
SMARD . . . 10
Energy-Charts (installierte Leistung) . . . 11
6 Phase 0: Datenprüfung (abgeschlossen) . . . 12
Wetter: Lückenprüfung über den ganzen Zeitraum . . . 12
SMARD . . . 12
Installierte Leistung: ein falscher Nenner . . . 13
Wetterorte . . . 13
Ergebnis . . . 16
7 Architektur . . . 17
Aufbau des Repositorys . . . 18
8 Phase 1: Daten laden und eigenes Archiv (läuft) . . . 19
Der Trainingsdownload Schritt für Schritt . . . 19
Das eigene Archiv . . . 19
9 Phase 2: Das Solar-Modell (geplant) . . . 22
Schritt 1: Trainingstabelle bauen . . . 22
Schritt 2: Merkmale . . . 22
Schritt 3: Training . . . 23
Schritt 4: Bewertung . . . 23

Die Messlatten . . . 24
10 Phase 3: Wind an Land und auf See (geplant) . . . 25
11 Phase 4: Das Tool in smard-mcp (geplant) . . . 26
Modelle von Python nach TypeScript . . . 26
Ablauf einer Anfrage . . . 26
Schnittstelle . . . 27
Verhalten am Rand . . . 27
12 Phase 5 und 6: Monitoring, Neu-Training, Ausrollen (geplant) . . . 28
Monitoring . . . 28
Monatliches Neu-Training . . . 28
Ausrollen und Prüfpunkt C . . . 28
13 Betrieb und Kosten . . . 29
Wo man nachsieht . . . 29
14 Risiken und offene Fragen . . . 30
Offene Fragen . . . 30
15 Entstehung: das Projekttagebuch . . . 31
Zeitleiste . . . 31
16 Glossar . . . 32

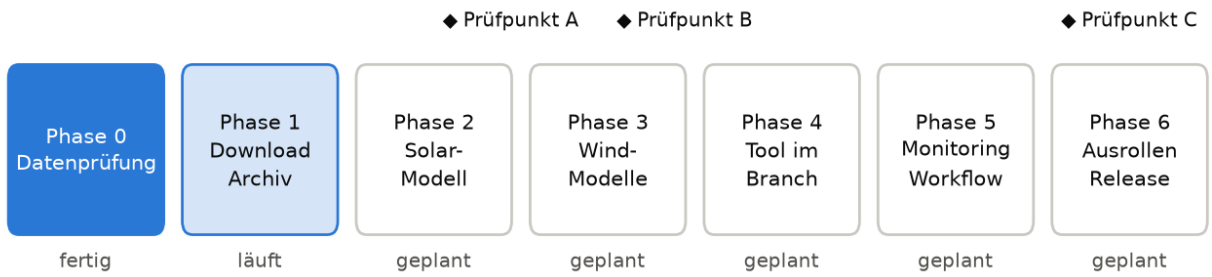
1 Kurzfassung

smard-mcp ist ein öffentlicher MCP-Server, über den KI-Assistenten deutsche Strommarktdaten von SMARD (Bundesnetzagentur) abfragen. Er beantwortet heute Fragen über Vergangenheit und Gegenwart: wie viel Solarstrom gestern, wie teuer morgen. Was fehlt, ist der Blick nach vorn über den nächsten Tag hinaus. Die Netzbetreiber veröffentlichen ihre Prognose nur für den Folgetag.

smard-forecast soll diese Lücke schließen: ein neues Tool `forecast_renewables`, das für Solar, Wind an Land und Wind auf See die Erzeugung der nächsten 6 Tage liefert, und zwar nicht als eine Zahl, sondern als Bandbreite: p10 (in 9 von 10 Fällen wird es mehr), p50 (der mittlere Wert) und p90 (in 9 von 10 Fällen wird es weniger).

Grundlage sind Wettervorhersagen des Deutschen Wetterdienstes (Modell ICON), bezogen über Open-Meteo, und die tatsächliche Erzeugung aus SMARD. Ein Lernverfahren (LightGBM) lernt aus gut zweieinhalb Jahren Daten, wie aus einer Wettervorhersage mit einem bestimmten Vorlauf eine Stromerzeugung wird, und wie unsicher das ist.

> **Stand am 2. November 2026**
> **Phase 0 (Datenprüfung): abgeschlossen.** Die Daten tragen den Plan. Trainingsbeginn 22.02.2024, rund 32 Monate, Vorhersagen bis Tag 6. Tag 7 gibt es im fremden Archiv nicht.
> **Phase 1 (Daten laden, eigenes Archiv): läuft.** Der Download der Trainingsdaten läuft seit 18:37 UTC und braucht wegen der Abrufgrenzen etwa zwei Tage. Das eigene Archiv der Vorhersagen ist seit 18:41 UTC auf der Web-Maschine in Betrieb.
> **Phase 2 bis 6: geplant.** Code in smard-mcp entsteht erst, wenn das Solar-Modell die Messlatten schlägt (Prüfpunkt A).



Stand 02.11.2026. Erst wenn Prüfpunkt A (Solar schlägt die Messlatten) bestanden ist, wird Code in smard-mcp eingebaut.

Abbildung 1: Die sieben Phasen und drei Prüfpunkte. Blau: erledigt bzw. laufend.

Die wichtigsten Befunde bisher

- **Tag 7 fehlt im Archiv vollständig.** ICON global rechnet nur 180 Stunden voraus; ein Lauf von vor 7 Tagen erreicht den Zieltag nicht mehr. Version 1 geht deshalb bis Tag 6.
- **Die Daten sind fast lückenlos.** 40 Orte, 6 Vorlauftage, 33 Monate: ein einziger Ausfall im April 2026 (wenige Tage, nur einige Vorlauftage).
- **SMARD führt die installierte Leistung zu niedrig.** 2026: 103 statt 120 GW Solar, 9,2 statt 11,7 GW Wind auf See. Die Auslastung wird deshalb mit Monatswerten berechnet.
- **Open-Meteo zählt große Abrufe mehrfach.** Ein Abruf über ein Jahr mit vielen Spalten zählt 30- bis 150-fach. Der Download muss deshalb über zwei Tage verteilt werden.

2 Ausgangslage und Ziel

smard-mcp heute

smard-mcp (Version 1.3.0) läuft auf der Web-Maschine unter energie.akquisio.eu/mcp und als npm-Paket. Er bietet 11 Tools, unter anderem `get_current_status`, `get_generation_mix`, `analyze_prices` und `plan_charging`. Alle Tools lesen nur, rechnen in Berliner Zeit, nennen die Quelle und geben Felder wie `covered_until`, `complete`, `notes` und `warnings` zurück.

Was fehlt

Für die Zukunft gibt es bei SMARD nur zwei Dinge: die Day-Ahead-Prognose der Netzbetreiber (für morgen) und die Preise der Day-Ahead-Auktion (ab etwa 12 Uhr für morgen). Wer wissen will, ob am Wochenende viel Solarstrom kommt, ob eine Windflaute droht oder wann sich das Laden eines Elektroautos in den nächsten Tagen lohnt, bekommt keine Antwort.

Ziel

- Ein Tool `forecast_renewables` für Deutschland gesamt (Regelzonen später).
- Horizont: bis 6 Tage ab jetzt, stündlich oder täglich.
- Ausgabe: p10, p50, p90 in GW, Tagesenergie in GWh mit Bandbreite.
- Ehrlichkeit: das Tool nennt seinen eigenen gemessenen Fehler je Vorlauftag.
- Betrieb ohne GPU, ohne Kosten, ohne neue Abhängigkeiten im Server.

Was ausdrücklich nicht dazugehört (Version 1)

- Anteil an der Last: bräuchte eine eigene Verbrauchsprognose.
- Regelzonen (50Hertz, Amprion, TenneT, TransnetBW).
- Preisprognose.
- Tag 7: erst, wenn das eigene Archiv ein Jahr umfasst.

3 Grundbegriffe

Dieses Kapitel erklärt die Begriffe, die im Rest des Berichts vorkommen. Wer sie kennt, kann es überspringen.

Wettermodell, Lauf und Vorlaufzeit

Ein numerisches Wettermodell rechnet aus dem aktuellen Zustand der Atmosphäre die Zukunft voraus. Jede Rechnung heißt **Lauf** und hat einen **Startzeitpunkt** (Initialisierung). Der Deutsche Wetterdienst betreibt drei ICON-Modelle:

Tabelle 1: Die ICON-Modelle des DWD

Modell	Gebiet	Auflösung	Läufe	reicht bis
ICON-D2	Deutschland und Nachbarn	2,2 km	alle 3 h	48 h
ICON-D2-RUC	wie D2	2,2 km	stündlich	kurz (Schnellaktualisierung)
ICON-EU	Europa	6,5 km	alle 3 h (lange Läufe alle 6 h)	120 h
ICON (global)	Welt	13 km	alle 6 h (lange Läufe 00 und 12 UTC)	180 h

Vorlaufzeit ist der Abstand zwischen Startzeitpunkt eines Laufs und der Stunde, für die er etwas vorhersagt. Je länger der Vorlauf, desto unsicherer die Vorhersage. `icon_seamless` ist eine Zusammenstellung von Open-Meteo: für die ersten Stunden die feinste verfügbare Quelle (D2), danach EU, danach global.

Quantile: p10, p50, p90

Eine Punktprognose sagt „morgen 12 Uhr: 40 GW Solar“. Eine **Quantilsprognose** sagt: „mit 10 % Wahrscheinlichkeit weniger als 31 GW (p10), mit 50 % weniger als 40 GW (p50), mit 90 % weniger als 46 GW (p90)“. Zwischen p10 und p90 sollte der echte Wert also in etwa 80 % aller Stunden liegen. Das nennt man **Trefferquote der Bandbreite** (englisch coverage).

Ein Modell lernt ein Quantil τ mit dem **Pinball-Verlust**. Ist der Fehler e = y minus Vorhersage, dann ist der Verlust τ·e bei e größer 0 und (τ − 1)·e bei e kleiner 0. Für τ = 0,9 kostet zu niedrig Vorhersagen neunmal so viel wie zu hoch, deshalb landet die Vorhersage so, dass nur 10 % der echten Werte darüber liegen.

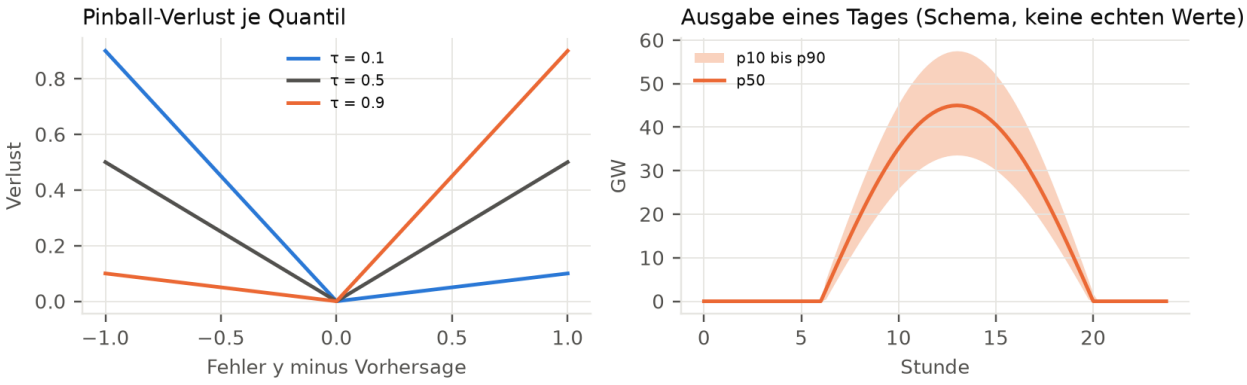


Abbildung 2: Links: Pinball-Verlust für drei Quantile. Rechts: so sieht eine Tagesausgabe aus (Schema mit erfundenen Werten).

Auslastung

Auslastung = Erzeugung geteilt durch installierte Leistung, ein Wert zwischen 0 und 1. Weil jeden Monat Anlagen dazukommen, wächst die Erzeugung bei gleichem Wetter. Lernt das Modell die Auslastung statt der GW, stört der Zubau nicht. Im Betrieb wird mit der aktuellen Leistung zurück in GW gerechnet.

LightGBM

LightGBM ist ein Verfahren aus Entscheidungsbäumen (Gradient Boosting). Viele kleine Bäume werden nacheinander gelernt, jeder korrigiert die Fehler der vorherigen. Vorteile hier: schnell auf der CPU, kommt mit vielen Spalten und fehlenden Werten klar, kann Quantile lernen, und ein fertiges Modell ist nur eine Liste von Bäumen, die sich in jeder Sprache mit ein paar Zeilen auswerten lässt.

4 Das Vorlaufzeit-Problem

Das ist der Kern des ganzen Projekts, deshalb ein eigenes Kapitel.

> **Die Regel**
> Ein Modell muss mit genau den Vorhersagen trainiert werden, die es im Betrieb bekommt. Eine Prognose für Tag 5 lernt man nur aus früheren Vorhersagen, die ebenfalls 5 Tage Vorlauf hatten.

Warum? Eine 5 Tage alte Wettervorhersage liegt oft daneben. Ein Modell, das mit fast perfektem Wetter trainiert wurde, vertraut der Vorhersage zu sehr und gibt zu schmale Bandbreiten aus. Im Test sähe es gut aus, im Betrieb wäre es schlecht.

Zwei Archive bei Open-Meteo

Tabelle 2: Die Archive von Open-Meteo

API	was sie liefert	geeignet?
Historical Forecast API	setzt aus jedem Lauf nur die ersten Stunden zusammen; also fast immer Vorlauf 0 bis 6 h, fast so gut wie eine Messung	nein, viel zu optimistisch
Previous Runs API	für jede Stunde zusätzlich die Werte aus dem Lauf von vor 1, 2, ... Tagen (Spalten `_previous_day1` bis `_previous_day7`)	ja, genau das Richtige
Single Runs API	einen ganzen Lauf mit exaktem Startzeitpunkt	nur ab Mai 2026, für Prüfungen

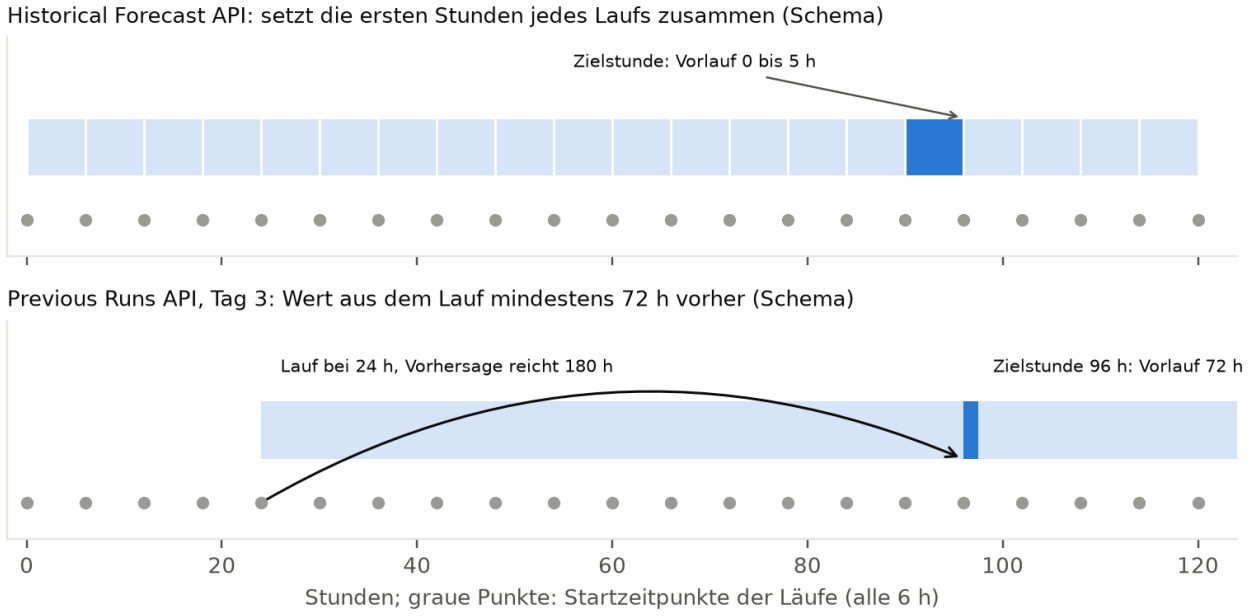


Abbildung 3: Oben: die Historical Forecast API nimmt für jede Stunde den jüngsten Lauf. Unten: die Previous Runs API nimmt für Tag 3 den Lauf, der mindestens 72 h vorher startete.

Was „Tag N“ genau bedeutet (geprüft)

Am 2. November wurde für einen Ort (52,5° N, 10° E) und den 20.10.2026 jede Stunde aus der Previous Runs API mit allen Einzelläufen der Woche davor verglichen, Wert für Wert (Temperatur und Wind in 100 m).

Tabelle 3: Abgleich mit Einzelläufen

Spalte	passt zu	Vorlauf
previous_day3	Lauf 3 Tage vorher, genau	72 bis 77 h
previous_day6	ICON-global-Lauf, genau	144 bis 150 h
previous_day1	keinem Einzellauf genau; Abweichung wenige Zehntel	etwa 24 bis 27 h

Daraus folgt die Definition, die in Training und Betrieb gleich gilt: **Tag N ist der neueste Lauf, der mindestens N mal 24 Stunden vor der Zielstunde gestartet ist.** Im Betrieb wird für jede Stunde die Vorlaufzeit = Zielstunde minus Startzeitpunkt des Laufs berechnet und auf ganze Tage abgerundet; Stunden unter 24 h zählen als Tag 1.

Dass Tag 1 nicht exakt passt, liegt sehr wahrscheinlich an ICON-D2-RUC: Open-Meteo nutzt für die ersten Stunden inzwischen die stündlich aktualisierte Variante, und die Single Runs API führt diese Lauffolge nicht vollständig. Für das Training ist das unkritisch. Das eigene Archiv speichert deshalb die Startzeitpunkte aller ICON-Modelle mit, damit sich das später genau klären lässt.

5 Datenquellen

Open-Meteo

Open-Meteo stellt die DWD-Modelle frei über eine einfache HTTP-Schnittstelle bereit. Die Daten stehen unter CC BY 4.0. Die freie Schnittstelle ist für nicht-kommerzielle Nutzung ohne Werbung und Abo gedacht; smard-mcp ist kostenlos und werbefrei.

Abrufgrenzen und Gewichtung

Open-Meteo erlaubt frei 600 Abrufe pro Minute, 5.000 pro Stunde und 10.000 pro Tag. Große Abrufe zählen mehrfach: anteilig mehr ab 10 Spalten und ab 14 Tagen, und jeder Ort zählt einzeln. Als Formel (so im Code, Funktion `gewicht()`):

```
gewicht = orte × max(1, spalten / 10) × max(1, tage / 14)
```

Beispiel: ein Solar-Ort, 6 Variablen × 6 Vorlauftage = 36 Spalten, ein Jahr (365 Tage): 1 × 3,6 × 26,1 = 94 Einheiten. Ein Ort, der für Solar und Wind zählt, hat 10 Variablen, also 60 Spalten: 156 Einheiten pro Jahr.

Variablen

Tabelle 4: Wettervariablen

Variable	Einheit	für	im Archiv
shortwave_radiation	W/m²	Solar	ja
direct_radiation	W/m²	Solar	ja
diffuse_radiation	W/m²	Solar	ja
cloud_cover	%	Solar	ja
temperature_2m	°C	Solar und Wind (Modulwärme, Luftdichte)	ja
wind_speed_10m	km/h	Solar (Kühlung) und Wind	ja
wind_speed_100m	km/h	Wind (Nabenhöhe)	ja, ab 16.02.2024
wind_direction_100m	°	Wind	ja
wind_gusts_10m	km/h	Wind (Sturmabschaltung)	ja
surface_pressure	hPa	Wind (Luftdichte)	ja
direct_normal_irradiance	W/m²	(ableitbar, nicht geladen)	ja
cloud_cover_low	%	geplant	nein
snow_depth	m	geplant (Schnee auf Modulen)	nein

SMARD

SMARD ist die Strommarktdaten-Plattform der Bundesnetzagentur. Die Daten kommen über `https://www.smard.de/app/chart_data` als Wochenstücke im JSON-Format. Die Kennungen sind dieselben wie in smard-mcp (src/catalog.ts).

Tabelle 5: SMARD-Reihen

Schlüssel	Filter-ID	Inhalt
generation_solar	4068	Erzeugung Photovoltaik
generation_wind_onshore	4067	Erzeugung Wind an Land
generation_wind_offshore	1225	Erzeugung Wind auf See
forecast_generation_solar	125	Day-Ahead-Prognose der Netzbetreiber, Solar
forecast_generation_wind_onshore	123	Prognose Wind an Land
forecast_generation_wind_offshore	3791	Prognose Wind auf See
capacity_solar / _wind_onshore / _wind_offshore	188 / 186 / 4076	installierte Leistung, nur Jahreswerte

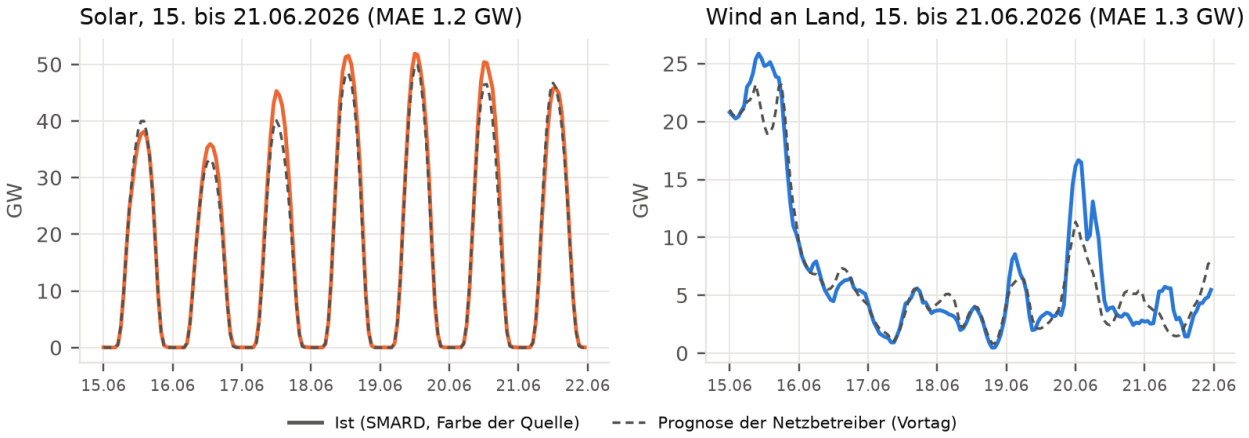


Abbildung 4: Eine Woche im Juni 2026: echte Erzeugung gegen die Prognose der Netzbetreiber vom Vortag. Das ist die Messlatte für Tag 1.

Energy-Charts (installierte Leistung)

Energy-Charts (Fraunhofer ISE) veröffentlicht die installierte Leistung monatlich, abgeleitet aus dem Marktstammdatenregister. Abgerufen über `api.energy-charts.info/installed_power?country=de&time_step=monthly`. Für Solar gibt es zwei Werte: DC (Modulleistung) und AC (Wechselrichterleistung). Genommen wird AC, weil die Wechselrichter die Einspeisung begrenzen.

Offener Punkt

Die Lizenz der Energy-Charts-Daten ist noch zu klären; die Seite mit den Nutzungshinweisen ließ sich ohne Browser nicht lesen. Rückfall: Monatssummen selbst aus dem Marktstammdatenregister berechnen (Lizenz dl-de/by-2-0).

6 Phase 0: Datenprüfung (abgeschlossen)

Ziel von Phase 0: bevor eine Zeile Modellcode entsteht, beweisen, dass die Daten den Plan tragen. Ein Fehlschlag hier kostet nur Zeit, nicht smard-mcp.

Wetter: Lückenprüfung über den ganzen Zeitraum

Skript `scripts/phase0_wetter.py`. Für alle 40 Orte, Zeitraum 25.01.2024 bis 01.11.2026, je Vorlauftag 1 bis 6: Temperatur, dazu Strahlung (Solar-Orte) bzw. Wind in 100 m (Wind-Orte). Je Ort drei Abrufe (je Kalenderjahr), jede Antwort wird gespeichert.

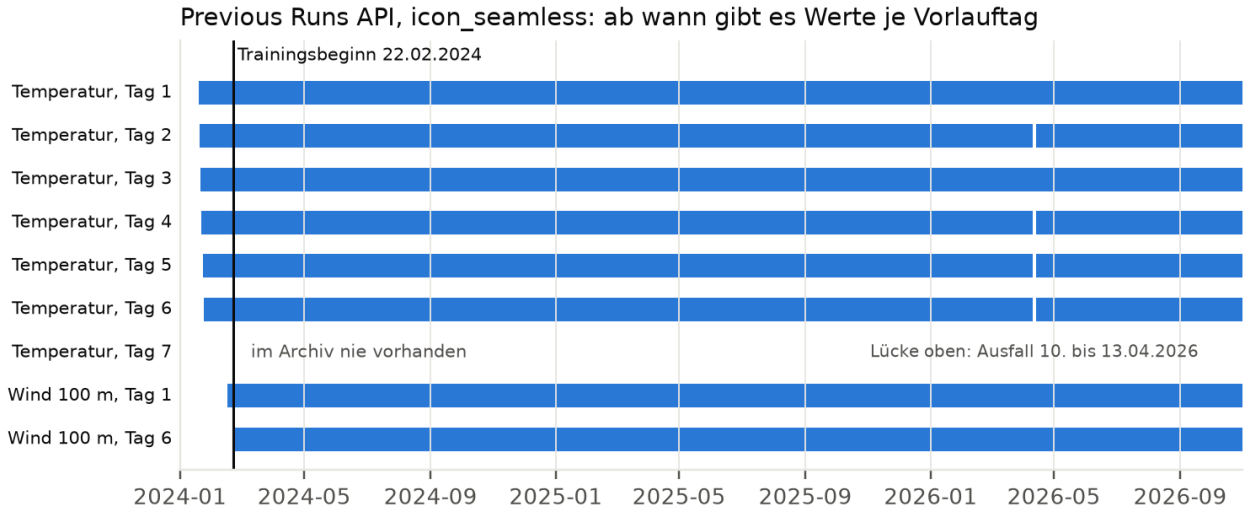


Abbildung 5: Ab wann es Werte gibt, je Variable und Vorlauftag. Die senkrechte Linie ist der gewählte Trainingsbeginn.

Tabelle 6: Befunde der Wetterprüfung

Befund	Umfang	Folge
Ausfall im Archiv	10. bis 13.04.2026, Tag 2, 4, 5, 6, an allen Orten gleich; Tag 1 und 3 vollständig	Stunden fallen im Training weg
wind_speed_100m beginnt später	Tag 1 ab 16.02.2024, Tag 6 ab 21.02.2024	Trainingsbeginn 22.02.2024 für alle Quellen
previous_day7	nie vorhanden	Version 1 bis Tag 6
cloud_cover_low, snow_depth	nicht im Archiv	Merkmale entfallen
Offshore-Orte	alle 7 auf 0 m Höhe	liegen wirklich auf See
sonst	keine Lücken	Rückfallebene nicht nötig

SMARD

Skript `scripts/smard_laden.py` (in Phase 0 noch phase0_smard.py). Erzeugung aller drei Quellen ab 01.01.2024 stündlich: lückenlos. Netzbetreiber-Prognose: ein fehlender Tag (23.10.2026 22 Uhr bis 24.10.2026, Solar und Wind an Land).

Installierte Leistung: ein falscher Nenner

Prüfidee: Wenn die installierte Leistung stimmt, sollte die höchste Auslastung über die Jahre etwa gleich bleiben (das Wetter hat keinen Trend). Mit den SMARD-Jahreswerten fällt sie bei Solar, und bei Wind auf See steigt sie. Der Vergleich mit Energy-Charts zeigt den Grund: SMARD hinkt dem Zubau hinterher.

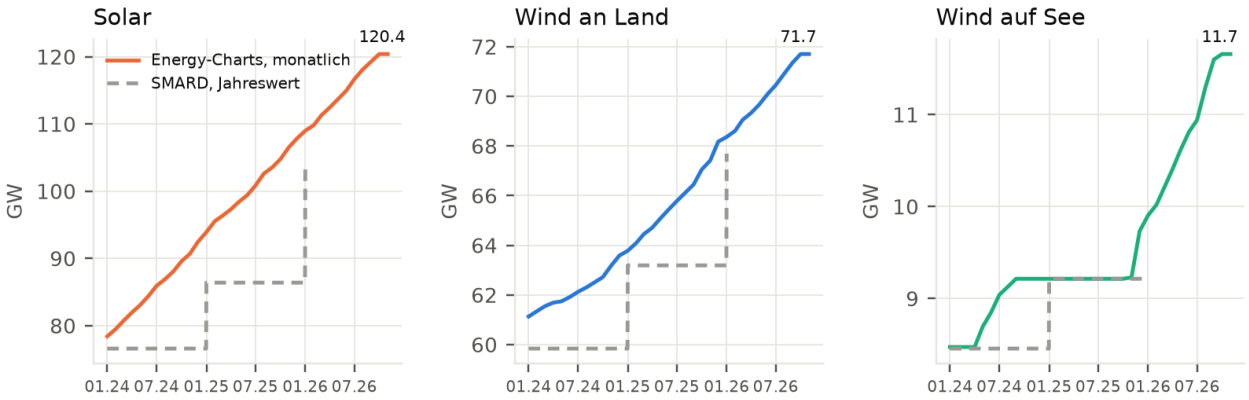


Abbildung 6: Installierte Leistung: SMARD-Jahreswerte (gestrichelt) gegen Monatswerte von Energy-Charts. Zahl am Ende: Stand Oktober 2026 in GW.

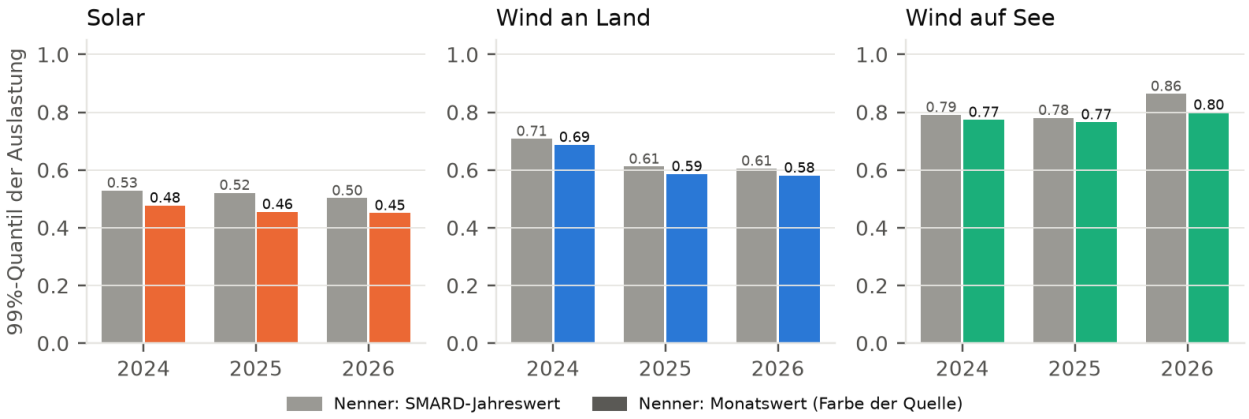


Abbildung 7: 99%-Quantil der stündlichen Auslastung je Jahr, einmal mit SMARD als Nenner, einmal mit Monatswerten. Mit Monatswerten liegt die Auslastung niedriger und gleichmäßiger.

Zahlen 2026: Solar 103,3 GW (SMARD) gegen 120,4 GW AC (Energy-Charts), Wind an Land 67,7 gegen 71,7 GW, Wind auf See 9,2 gegen 11,7 GW. **Entscheidung:** Monatswerte als Nenner.

Wetterorte

40 feste Orte in `config/orte.toml`: 21 Solar-Orte nach Schwerpunkten der Photovoltaik (stark in Bayern und Baden-Württemberg), 14 Wind-Orte (Küste, Norddeutsches Tiefland, Mittelgebirge), 7 Orte auf See in den Clustern der Nord- und Ostsee. Magdeburg und Cottbus zählen für Solar und Wind. Eine Gewichtung aus dem Marktstammdatenregister kommt später; das Modell gewichtet die Orte ohnehin selbst.



Abbildung 8: Die 40 Wetterorte.

Tabelle 7: Alle Wetterorte

Name	Breite	Länge	Typ
muenchen	48.14	11.58	solar
augsburg	48.37	10.90	solar
kempten	47.73	10.31	solar
landshut	48.54	12.15	solar
regensburg	49.01	12.10	solar
nuernberg	49.45	11.08	solar
wuerzburg	49.79	9.95	solar
ulm	48.40	9.99	solar
stuttgart	48.78	9.18	solar
freiburg	47.99	7.85	solar
mannheim	49.49	8.47	solar
koeln	50.94	6.96	solar
muenster	51.96	7.63	solar
kassel	51.31	9.48	solar
hannover	52.37	9.73	solar
erfurt	50.98	11.03	solar
leipzig	51.34	12.37	solar
potsdam	52.40	13.06	solar
guestrow	53.79	12.17	solar
magdeburg	52.08	11.40	solar, wind
cottbus	51.76	14.33	solar, wind
husum	54.48	9.05	wind
dithmarschen	54.19	9.10	wind
fehmarn	54.45	11.15	wind
emden	53.37	7.21	wind
cuxhaven	53.86	8.69	wind
soltau	52.99	9.84	wind
altmark	52.60	11.86	wind
neubrandenburg	53.56	13.26	wind
prenzlau	53.32	13.86	wind
paderborn	51.72	8.75	wind
vogelsberg	50.60	9.25	wind
hunsrueck	49.98	7.52	wind
nordsee_borkum	53.98	6.50	offshore
nordsee_godewind	54.05	7.00	offshore
nordsee_helgoland	54.35	7.75	offshore
nordsee_deutsche_bucht	54.32	5.85	offshore
nordsee_sandbank	55.10	7.20	offshore
ostsee_baltic	54.80	13.00	offshore
ostsee_arkona	54.78	14.07	offshore

Ergebnis

Phase 0 bestanden
Trainingsbeginn 22.02.2024, rund 32 Monate, 6 Vorlauftage, 40 Orte. Einziger offener Punkt: Lizenz Energy-Charts.

7 Architektur

Das Projekt verteilt sich auf zwei Maschinen, zwei Repositories und drei externe Quellen.

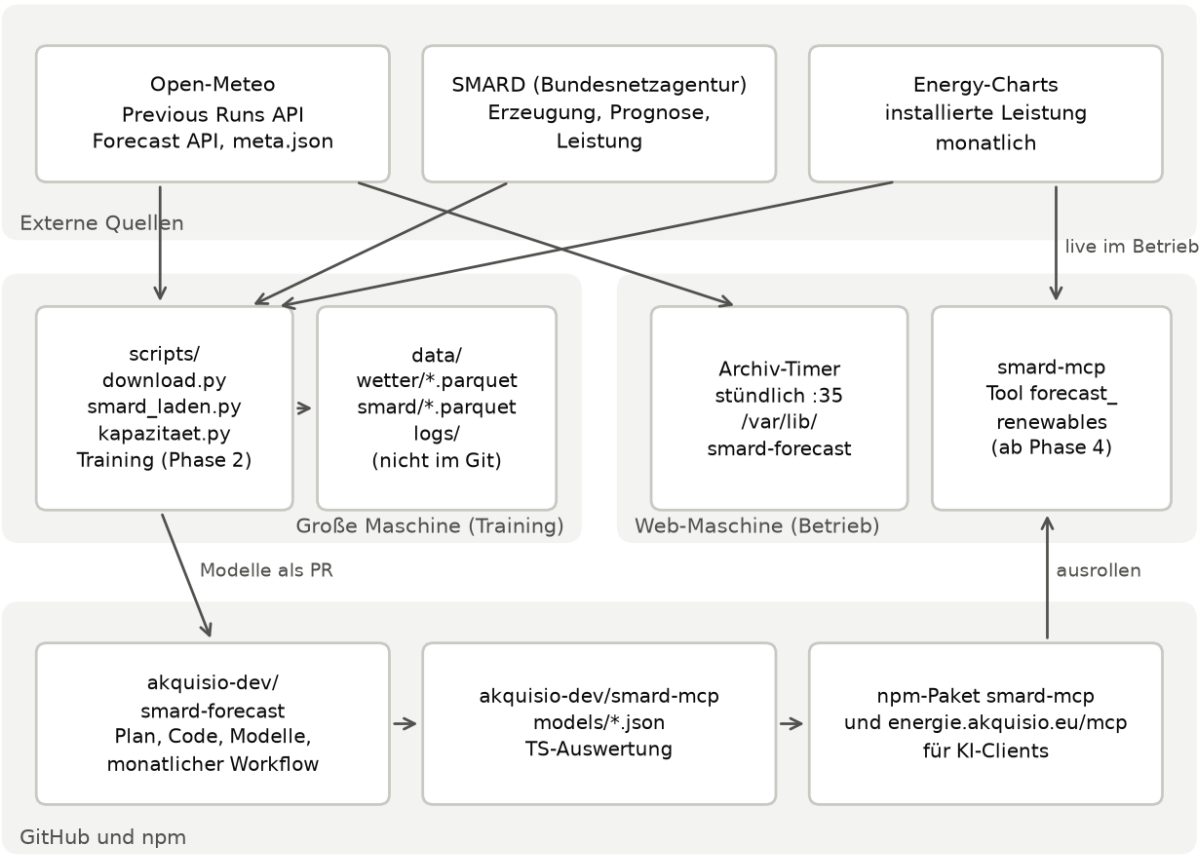


Abbildung 9: Wer was macht. Training auf der großen Maschine, Betrieb auf der Web-Maschine, Austausch über GitHub.

Tabelle 8: Orte im Überblick

Ort	Was dort liegt	Warum dort
große Maschine, /root/smard-forecast	Download, Trainingsdaten (data/), Training, Berichte	viel Arbeitsspeicher und Platz; Training ist rechenintensiver
Web-Maschine, /opt/smard-forecast-archiv	Archiv-Skript und Ortsliste	läuft dauerhaft, braucht kaum Ressourcen, smard-mcp läuft dort auch
Web-Maschine, /var/lib/smard-forecast/archiv	gespeicherte Vorhersagen	Zustand des Dienstes, nachts gesichert
GitHub akquisio-dev/smard-forecast	Plan, Code, später Modelle und Workflow	öffentlich, nachvollziehbar
GitHub akquisio-dev/smard-mcp	Tool, TS-Auswertung, models/*.json	Auslieferung
npm, energie.akquisio.eu/mcp	das fertige Tool	dort nutzen es KI-Clients

Aufbau des Repositorys

```
smard-forecast/
├── README.md, LICENSE (MIT), pyproject.toml, .gitignore
├── config/orte.toml          40 Wetterorte (Name, Breite, Länge, Typ)
├── sf/                       kleine Python-Bibliothek
│   ├── orte.py               lädt die Ortsliste
│   ├── openmeteo.py          Previous Runs API, Variablen je Typ, gewicht()
│   ├── smard.py              SMARD chart_data, Kennungen
│   └── log.py                Protokoll mit Zeitstempel nach data/logs/
├── scripts/
│   ├── phase0_wetter.py      Lückenprüfung Wetter
│   ├── smard_laden.py        SMARD laden und prüfen
│   ├── download.py           Trainingsdaten laden (Phase 1)
│   └── kapazitaet.py         installierte Leistung monatlich
├── archiv/
│   ├── archiv.py             Archiv-Job (Server, nur Standardbibliothek)
│   ├── smard-forecast-archiv.service / .timer
│   └── ausrollen.sh          bringt den Job auf den Server
├── docs/
│   ├── PLAN.md, phase0.md, phase1.md
│   ├── journal.md            Projekttagebuch
│   └── bericht/              dieser Bericht (abbildungen.py, bericht.py)
└── data/                     nicht im Git: wetter/, smard/, phase0/, logs/
```

Alles unter `data/` lässt sich jederzeit neu laden und ist deshalb nicht im Git. Ausnahme ist das Archiv auf dem Server: das lässt sich nicht neu beschaffen und wird deshalb gesichert.

8 Phase 1: Daten laden und eigenes Archiv (läuft)

Der Trainingsdownload Schritt für Schritt

Skript `scripts/download.py`, gestartet als eigener systemd-Dienst `smard-forecast-download`, damit er unabhängig von der Sitzung weiterläuft.

1. Ortsliste lesen; je Ort die Variablen bestimmen (Solar 6, Wind 6, beides 10).
2. Zeitraum 22.02.2024 bis 01.11.2026 in Kalenderjahre teilen: 3 Stücke je Ort, 120 insgesamt.
3. Stücke überspringen, deren Datei schon existiert (`data/wetter/<ort>/<von>_<bis>.parquet`). Dadurch kann der Download jederzeit abbrechen und weitermachen.
4. Für jedes offene Stück das Gewicht berechnen (94 bis 156 Einheiten pro Jahr).
5. Prüfen, ob das Gewicht in alle drei Fenster passt: Tag (höchstens 9.000), letzte Stunde (4.500), letzte Minute (450). Die Grenzen liegen bewusst unter denen von Open-Meteo. Wenn nicht, warten: bei vollem Tag bis 00:05 UTC, bei voller Stunde je 2 Minuten, bei voller Minute je 15 Sekunden.
6. Abruf an die Previous Runs API; bei leerer Antwort, Zeitüberschreitung oder Fehler 429/5xx bis zu fünfmal wiederholen, mit wachsender Pause.
7. Abruf mit Zeitstempel und Gewicht in `data/wetter/budget.json` buchen.
8. Antwort in die lange Form bringen: eine Zeile je Stunde und Vorlauftag, Spalten zeit (UTC), vorlauf (1 bis 6) und die Variablen als Float32.
9. Erst in eine .tmp-Datei schreiben, dann umbenennen: eine halb geschriebene Datei kann so nie als fertig gelten.
10. Zeile ins Protokoll `data/logs/download.log`: Ort, Zeitraum, Zeilen, ganz leere Zeilen, Gewicht.

Gesamtgewicht etwa 10.460 Einheiten. Am 2. November war das Tageskontingent durch die Lückenprüfung schon belastet, deshalb lädt der Download am ersten Abend etwa 4.500 Einheiten und den Rest ab 00:05 UTC am 3. November.

```
# Aufbau einer Trainingsdatei (Magdeburg); Schema, Werte erfunden
zeit                       vorlauf  shortwave_radiation  ...  wind_speed_100m  surface_pressure
2025-06-01 12:00:00+00:00  1        812.0                ...  19.4             1003.1
2025-06-01 12:00:00+00:00  2        790.0                ...  21.0             1002.7
...
2025-06-01 12:00:00+00:00  6        655.0                ...  17.8             1004.9
```

Das eigene Archiv

Warum ein eigenes Archiv, wenn Open-Meteo eins hat? Drei Gründe: Tag 7 gibt es dort nicht; das Monitoring braucht genau die Vorhersagen, die zum Zeitpunkt der Prognose vorlagen; und ein fremdes Archiv kann Lücken bekommen oder verschwinden.

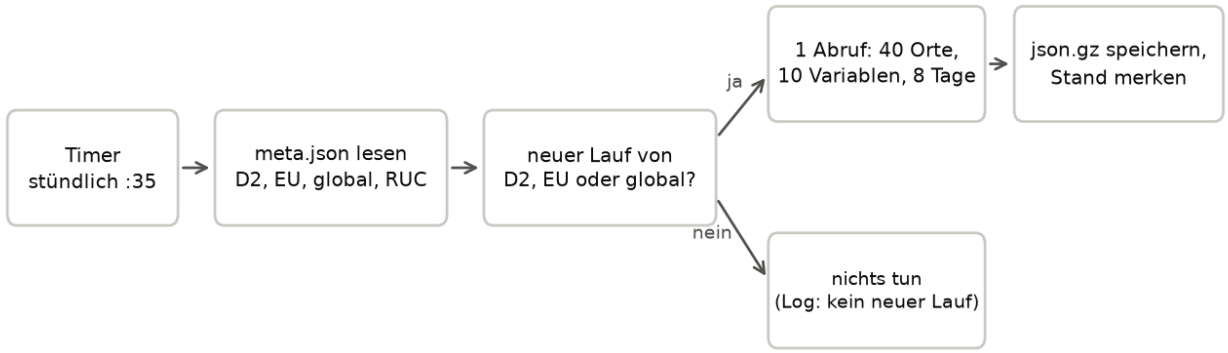


Abbildung 10: Was der Archiv-Job jede Stunde tut.

1. Der Timer startet den Dienst stündlich um :35 (mit bis zu 2 Minuten Zufallsversatz).
2. Der Dienst liest für vier Modelle `meta.json` von Open-Meteo und daraus `last_run_initialisation_time`. Das kostet kein Kontingent.
3. Hat sich der Lauf von ICON-D2, ICON-EU oder ICON global gegenüber `letzter_stand.json` geändert? RUC wird nur mitgeschrieben, sonst wären es 24 Abrufe am Tag.
4. Wenn ja: ein Abruf an die Forecast API mit allen 40 Orten kommagetrennt, 10 Variablen, 8 Tage (Gewicht 40).
5. Speichern als `/var/lib/smard-forecast/archiv/JJJJ/MM/TT/<zeit>.json.gz` mit Abrufzeit, Modell, allen Laufzeitpunkten und den Werten je Ort. Wieder über .tmp und Umbenennen.

```
{
  "abgerufen": "2026-11-02T18:41:29Z",
  "modell": "icon_seamless",
  "laeufe": {"dwd_icon_d2": "2026-11-02T15:00Z", "dwd_icon_eu": "2026-11-02T15:00Z",
             "dwd_icon": "2026-11-02T12:00Z", "dwd_icon_d2_ruc_native_15min": "2026-11-02T17:00Z"},
  "orte": [{"name": "muenchen", "lat": 48.14, "lon": 11.58, "elevation": 519.0,
            "hourly": {"time": [...192 Stunden...], "shortwave_radiation": [...], ...}}, ...]
}
```

Größe: etwa 95 KB je Abruf, etwa 8 Abrufe am Tag, rund 300 MB im Jahr.

Absicherung des Dienstes

Der Dienst läuft als eigener Systemnutzer `smard-forecast` ohne Anmeldung, mit systemd-Härtung: kein Schreiben außerhalb seines Zustandsverzeichnisses (`ProtectSystem=strict`, `StateDirectory`), kein Zugriff auf Home-Verzeichnisse, keine Rechte (`CapabilityBoundingSet=` leer), nur IPv4/IPv6, höchstens 256 MB Speicher. Der Server hat nur Python 3.10, deshalb nutzt das Skript nur die Standardbibliothek; die Ortsliste wird beim Ausrollen als JSON erzeugt.

Sicherung

Die nächtliche Dienste-Sicherung (04:00 UTC) holt das Archiv per ssh als tar.gz auf die große Maschine (Posten `smard-forecast`), prüft es und rotiert die Stände.

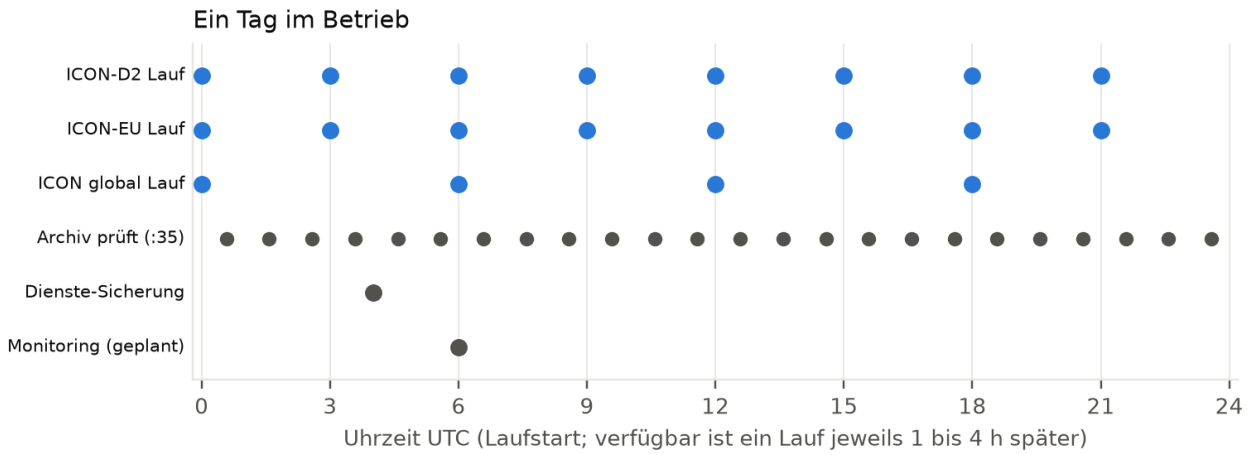


Abbildung 11: Ein Tag im Betrieb: Modellläufe, Archiv-Prüfungen, Sicherung.

9 Phase 2: Das Solar-Modell (geplant)

Solar kommt zuerst, weil es am besten vorhersagbar ist und der Fehler am klarsten zu messen. Schlägt Solar die Messlatten nicht, lohnt sich der Rest nicht.

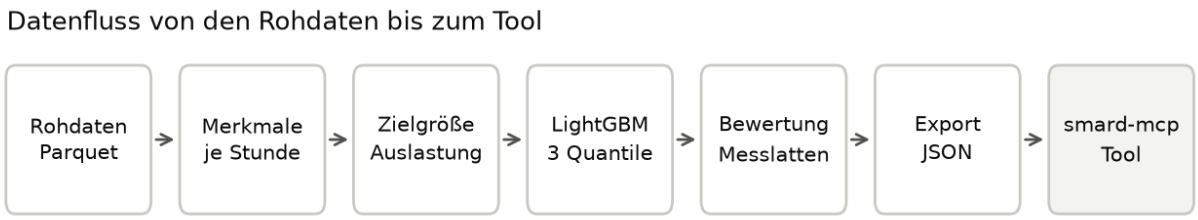


Abbildung 12: Datenfluss von den Rohdaten bis zum Tool.

Schritt 1: Trainingstabelle bauen

1. Alle Parquet-Stücke der Solar-Orte einlesen.
2. Breit machen: je (Zeit, Vorlauftag) eine Zeile; je Ort und Variable eine Spalte, zum Beispiel `muenchen__shortwave_radiation`. Bei 21 Orten und 6 Variablen sind das 126 Wetterspalten.
3. SMARD-Erzeugung Solar dazu (stündlich, UTC).
4. Zielgröße: Erzeugung geteilt durch installierte AC-Leistung des Monats.
5. Zeilen ohne Zielwert oder mit ganz leerem Wetter (Ausfall April 2026) entfernen.

Größe: rund 23.000 Stunden × 6 Vorlauftage = etwa 140.000 Zeilen, davon etwa die Hälfte am Tag.

Schritt 2: Merkmale

Tabelle 9: Merkmale des Solar-Modells

Merkmal	Berechnung	Zweck
Wetter je Ort	6 Variablen × 21 Orte	Hauptinformation; das Modell gewichtet selbst
Mittel über Orte	z. B. mittlere Strahlung aller Solar-Orte	stabiler Gesamtwert
vorlauf	1 bis 6	Unsicherheit wächst mit dem Vorlauf
Stunde	Sinus und Kosinus von 2π·h/24	Tagesgang ohne Sprung um Mitternacht
Tag im Jahr	Sinus und Kosinus von 2π·d/365,25	Jahresgang
Sonnenhöhe	berechnet für 51° N, 10° E (nicht vorhergesagt)	Geometrie, Einfallswinkel
Wochentag, Feiertag	bundesweite Feiertage	Abregelung bei negativen Preisen

Nachtstunden (Sonnenhöhe unter 0°) bekommen fest 0 und gehen weder ins Training noch in die Kennzahlen ein. Sonst sähe der Fehler künstlich klein aus, weil nachts jedes Modell recht hat.

Schritt 3: Training

Je Quantil ein Modell, also drei Modelle für Solar. Startwerte (werden in Phase 2 abgestimmt):

Tabelle 10: Startwerte LightGBM

Parameter	Startwert	Bedeutung
objective / alpha	quantile / 0,1, 0,5, 0,9	Pinball-Verlust für das Quantil
num_leaves	15	Größe eines Baums; klein hält das Paket klein
learning_rate	0,05	Schrittweite
n_estimators	höchstens 300, frühes Stoppen	Zahl der Bäume
min_data_in_leaf	50	gegen Auswendiglernen seltener Stunden
feature_fraction	0,8	je Baum nur ein Teil der Spalten
bagging_fraction / freq	0,8 / 1	je Baum nur ein Teil der Zeilen

Danach werden die drei Vorhersagen je Stunde sortiert, damit nie p10 über p50 oder p50 über p90 liegt (getrennt gelernte Quantile können sich sonst kreuzen). Werte unter 0 werden 0, Auslastung über 1 wird 1.

Schritt 4: Bewertung

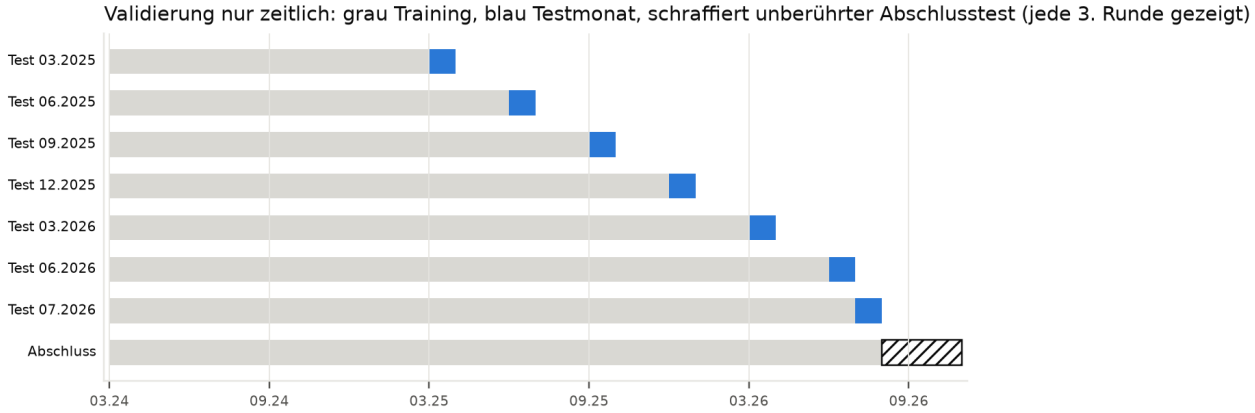


Abbildung 13: Zeitliche Validierung: trainieren bis Monat M, testen auf M+1; dazu die letzten 3 Monate als unberührter Abschlusstest.

Nie zufällig teilen: benachbarte Stunden sind fast gleich, eine zufällige Teilung würde das Modell mit der Antwort im Training testen. Die erste Testrunde beginnt nach 12 Monaten Training, damit jede Jahreszeit einmal gesehen wurde.

Tabelle 11: Kennzahlen, je Vorlauftag

Kennzahl	Formel	Ziel
MAE	Mittel von |echt − p50| in GW	klein
MAE in %	MAE geteilt durch installierte Leistung	vergleichbar über Jahre
Fehler Tagesenergie	|Summe echt − Summe p50| je Tag in GWh	für Tagesfragen
Trefferquote	Anteil Stunden mit p10 ≤ echt ≤ p90	um 80 %
Pinball	mittlerer Pinball-Verlust über alle drei Quantile	klein
Skill	1 − MAE Modell / MAE Messlatte	größer 0,2 ab Tag 2

Die Messlatten

Tabelle 12: Messlatten

Messlatte	was sie ist	gilt für
Klimatologie	mittlere Auslastung je Stunde und Monat der Vorjahre	Tag 1 bis 6
Persistenz	wie der letzte vollständige Tag	Tag 1 bis 6
Netzbetreiber	SMARD forecast_generation_* vom Vortag	Tag 1

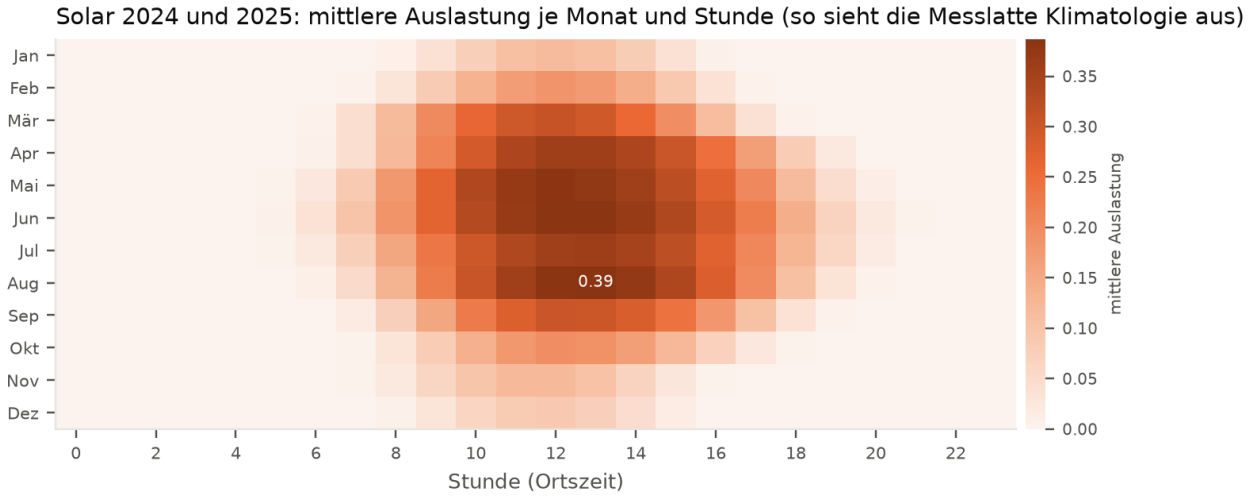


Abbildung 14: So sieht die Messlatte Klimatologie für Solar aus, aus echten Daten 2024 und 2025.

Prüfpunkt A: Weiter-Kriterium (Startwert)

An Tag 2 bis 6 mindestens 20 % weniger MAE als die bessere der beiden einfachen Messlatten.
An Tag 1 höchstens 20 % mehr Fehler als die Netzbetreiber.
Trefferquote der Bandbreite zwischen 75 und 85 %.
Die Wintermonate werden getrennt ausgewiesen, weil der Abschlusstest (August bis Oktober 2026) keinen Winter enthält.

10 Phase 3: Wind an Land und auf See (geplant)

Gleicher Ablauf wie Solar, mit anderen Merkmalen.

Tabelle 13: Unterschiede der Wind-Modelle

	Wind an Land	**Wind auf See**
Orte	16 (14 Wind + Magdeburg, Cottbus)	7 (5 Nordsee, 2 Ostsee)
Variablen	Wind 10 m und 100 m, Richtung, Böen, Temperatur, Luftdruck	dieselben
zusätzlich	Windrichtung als Sinus/Kosinus	dito
Besonderheit	Leistungskurve: unter 3 m/s nichts, ab etwa 12 m/s voll, ab etwa 25 m/s Abschaltung	wenige große Parks; Netzengpässe und Abregelung häufig
Nachtstunden	zählen voll	zählen voll

Prüfpunkt B: gleiche Kriterien wie A, je Quelle. Wind ist schwerer vorherzusagen als Solar; Wind auf See leidet zusätzlich unter Abregelung, die nichts mit dem Wetter zu tun hat.

11 Phase 4: Das Tool in smard-mcp (geplant)

Modelle von Python nach TypeScript

smard-mcp ist in TypeScript geschrieben und soll keine neue Abhängigkeit bekommen. LightGBM kann ein Modell als JSON ausgeben (`dump_model()`). Ein eigenes, kompaktes Format hält die Dateien klein:

```
// models/solar_p50.json (vereinfacht)
{ "version": "2026-12-01", "quelle": "solar", "quantil": 0.5,
  "spalten": ["vorlauf", "stunde_sin", ..., "muenchen__shortwave_radiation", ...],
  "basis": 0.0812,
  "baeume": [ { "s": [12, 3, ...],        // Spalte je innerem Knoten
                "t": [210.5, 0.31, ...],  // Schwelle
                "l": [1, 3, ...], "r": [2, 4, ...],  // Kinder; negativ = Blatt -(i+1)
                "v": [0.0123, -0.004, ...],          // Blattwerte
                "n": [0, 1, ...] } ] }    // fehlender Wert geht links (0) oder rechts (1)
```

```
// TypeScript: ein Baum, eine Zeile
function baum(b: Baum, x: Float64Array): number {
  let k = 0;
  for (;;) {
    const w = x[b.s[k]];
    const links = Number.isNaN(w) ? b.n[k] === 0 : w <= b.t[k];
    const z = links ? b.l[k] : b.r[k];
    if (z < 0) return b.v[-z - 1];
    k = z;
  }
}
const vorhersage = (m: Modell, x: Float64Array) =>
  m.baeume.reduce((s, b) => s + baum(b, x), m.basis);
```

Ein Test in smard-mcp vergleicht die TypeScript-Auswertung mit Python-Vorhersagen auf festen Beispielzeilen (Abweichung kleiner 1e-9). Ziel für alle neun Modelle zusammen: unter 2 MB.

Ablauf einer Anfrage

1. Eingaben prüfen: Zeitraum höchstens 6 Tage ab jetzt, nicht in der Vergangenheit.
2. Wetter holen: ein Abruf für alle 40 Orte, Ergebnis im Zwischenspeicher, bis `meta.json` einen neueren Lauf meldet (etwa 8 Abrufe am Tag).
3. Je Zielstunde die Vorlaufzeit bestimmen (Zielstunde minus Laufstart, auf Tage abgerundet).
4. Merkmale bauen, genau wie im Training (gemeinsame Spaltenliste aus der Modelldatei).
5. Drei Modelle je Quelle auswerten, sortieren, auf 0 bis 1 begrenzen.
6. Mal installierte Leistung (aktueller Monat) ergibt GW; je Tag summieren zu GWh.
7. Wo für morgen veröffentlicht: Netzbetreiber-Prognose daneben stellen.
8. Ausgabe mit Metadaten und den gemessenen Fehlern.

Schnittstelle

Tabelle 14: Eingaben von forecast_renewables

Parameter	Werte	Standard
from, to	ISO-Datum oder Datum mit Uhrzeit (Berlin), höchstens 6 Tage ab jetzt	jetzt bis +6 Tage
sources	solar, wind_onshore, wind_offshore	alle drei
resolution	hour, day	day, bei bis zu 48 h hour
include_intervals	ja oder nein	nein

```
// Beispielantwort, gekürzt; Zahlen erfunden
{ "period": {"from": "2026-12-03T00:00+01:00", "to": "2026-12-09T00:00+01:00"},
  "region": "DE", "resolution": "day",
  "sources": {
    "solar": { "days": [ {"date": "2026-12-03", "p10_gwh": 41, "p50_gwh": 58, "p90_gwh": 72,
                          "lead_day": 1, "official_forecast_gwh": 55}, ... ] },
    "wind_onshore": { ... }, "wind_offshore": { ... } },
  "model_version": "2026-12-01", "trained_until": "2026-11-30",
  "weather_run": {"dwd_icon_d2": "2026-12-02T21:00Z", "dwd_icon": "2026-12-02T18:00Z"},
  "skill": {"solar": {"mae_pct_by_lead_day": [2.1, 2.6, 3.0, 3.4, 3.8, 4.1],
                      "coverage_last_30_days": 0.79}},
  "notes": ["Tag 6 stammt aus ICON global (13 km)."],
  "warnings": [],
  "source": "Bundesnetzagentur | SMARD.de (CC BY 4.0); Wetterdaten: Open-Meteo.com (CC BY 4.0), "
            "Modelle des DWD",
  "server_time": "2026-12-02T23:10:04+01:00" }
```

Verhalten am Rand

Tabelle 15: Grenzfälle

Fall	Verhalten
Zeitraum über 6 Tage hinaus	Fehler, der den letzten möglichen Zeitpunkt nennt
Zeitraum in der Vergangenheit	Fehler mit Hinweis auf get_generation_mix
Open-Meteo nicht erreichbar	zuletzt gespeicherte Vorhersage mit warnings und Alter; ab 12 Stunden Alter ein Fehler
Netzbetreiber-Prognose fehlt	official_forecast weggelassen, Hinweis in notes
Feiertag, sonniges Wochenende	Hinweis auf mögliche Abregelung in notes

12 Phase 5 und 6: Monitoring, Neu-Training, Ausrollen (geplant)

Monitoring

Ein täglicher Job (geplant 06:00 UTC) nimmt die archivierten Vorhersagen, rechnet mit dem aktuellen Modell die Prognose, die das Tool damals ausgegeben hätte, und vergleicht sie mit den inzwischen veröffentlichten SMARD-Werten. Der Fehler der letzten 30 Tage je Vorlauftag erscheint in der Tool-Ausgabe. So sieht jeder Nutzer, wie verlässlich die Zahlen gerade sind, und nicht nur, wie gut sie im Test waren.

Monatliches Neu-Training

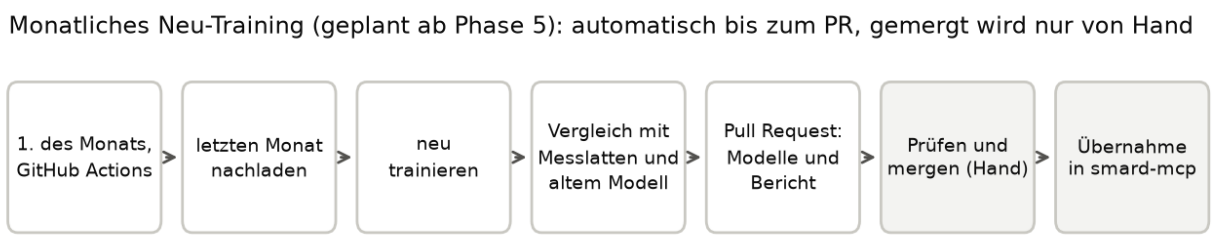


Abbildung 15: Automatisch bis zum Pull Request, gemergt wird nur von Hand.

Ein GitHub-Workflow lädt am Monatsanfang den letzten Monat nach, trainiert neu, misst gegen die Messlatten und gegen das bisherige Modell und öffnet einen Pull Request mit neuen Modelldateien und Fehlerbericht. Ist das neue Modell schlechter, steht das im Bericht; gemergt wird nie automatisch.

Ausrollen und Prüfpunkt C

1. Tool auf einem Branch in smard-mcp, Tests grün, TS gegen Python geprüft.
2. Ausrollen nur mit `npm run deploy` (sauberer, gepushter Stand, Tests, Sicherung, Health-Check, automatisches Zurückrollen).
3. 2 bis 4 Wochen den Fehler im Betrieb beobachten (Monitoring).
4. **Prüfpunkt C:** Fehler im Betrieb ähnlich wie im Abschlusstest? Dann Tag und npm-Release, sonst nachbessern.

13 Betrieb und Kosten

Tabelle 16: Laufende Kosten

Was	Wie oft	Kosten
Archiv-Job	stündlich prüfen, etwa 8 Abrufe am Tag	320 Einheiten am Tag, 300 MB im Jahr
Wetter für das Tool	wenn neuer Lauf, etwa 8 Abrufe am Tag	320 Einheiten am Tag
Tool-Anfrage	je Anfrage	Millisekunden Rechenzeit
Monitoring	täglich	Sekunden
Neu-Training	monatlich, GitHub Actions	Minuten CPU, unter 1 GB RAM
Erstdownload	einmalig	10.460 Einheiten, 2 Tage

Alles zusammen bleibt weit unter 10.000 Einheiten am Tag. Es fallen keine Geldkosten an.

Wo man nachsieht

Tabelle 17: Nützliche Befehle

Frage	Befehl
Läuft der Download?	`systemctl status smard-forecast-download`
Fortschritt Download	`tail -5 /root/smard-forecast/data/logs/download.log`
Archiv-Timer (Web-Maschine)	`systemctl list-timers smard-forecast-archiv.timer`
Letzte Archiv-Läufe	`journalctl -u smard-forecast-archiv -n 20`
Archiv-Dateien	`ls /var/lib/smard-forecast/archiv/2026/11/`
Bericht neu bauen	`docs/bericht/abbildungen.py` dann `bericht.py`

14 Risiken und offene Fragen

Tabelle 18: Risiken

Risiko	Folge	Gegenmaßnahme
Lücken im ICON-Archiv	weniger Trainingsdaten	geprüft: nur ein Ausfall; eigenes Archiv
Abregelung bei negativen Preisen	Erzeugung unter dem Wetterwert, vor allem Solar an Feiertagen	Wochentag und Feiertag als Merkmal, Hinweis in notes
Zubau	systematischer Versatz	Monatswerte statt SMARD-Jahreswerte
Seltene Wetterlagen (Dunkelflaute, Sturm)	grobe Ausreißer	Quantile, Winter getrennt bewertet
Open-Meteo ändert Bedingungen oder fällt aus	keine Prognose	Zwischenspeicher mit warnings, langfristig DWD Open Data als zweite Quelle
Wechsel in icon_seamless (z. B. D2-RUC)	Verteilung der Eingaben ändert sich	Laufzeitpunkte im Archiv, Monitoring zeigt Brüche
Lizenz Energy-Charts	Nenner fehlt	Rückfall Marktstammdatenregister
Lizenz Open-Meteo	nur nicht-kommerziell frei	smard-mcp bleibt kostenlos und werbefrei, sonst API-Key-Tarif

Offene Fragen

- Lizenz der Energy-Charts-Daten.
- Welche Lauffolge liefert icon_seamless in den ersten 48 Stunden genau (D2 oder D2-RUC)?
- Bundesweite Feiertage oder gewichtet nach Bundesländern?
- Ab wann Tag 7 aus dem eigenen Archiv (frühestens November 2027)?

15 Entstehung: das Projekttagebuch

Das vollständige Tagebuch steht in `docs/journal.md`. Hier die Fehlschläge und was daraus gelernt wurde, weil man daraus am meisten mitnimmt.

Tabelle 19: Fehlschläge und Lehren

Zeit (UTC)	Was passiert ist	Ursache	Lösung, Lehre
02.11. 08:00	Doku sagt „ab Januar 2024“, Tag 7 fehlt aber ganz	ICON global reicht nur 180 h	Riskanteste Annahme zuerst gegen die echte Quelle prüfen
02.11. 08:20	Tag 1 passt zu keinem Einzellauf	vermutlich ICON-D2-RUC	Laufzeitpunkte aller Modelle archivieren
02.11. 08:25	Single Runs API wäre ideal, beginnt aber erst Mai 2026	Archivtiefe	nur für Prüfungen nutzen
02.11. 08:35	Auslastung zeigt Scheintrend	SMARD-Leistung zu niedrig	Plausibilität über Jahre prüfen; Monatswerte
02.11. 08:50	Lückenprüfung bricht nach 11 Orten ab (leere Antwort)	Minutenlimit: 3 s Abstand ergab rund 620 Einheiten/min	Gewichte rechnen, 10 s Abstand, jede Antwort speichern
02.11. 18:30	Ausrollen auf die Web-Maschine abgelehnt	Freigabe-Prüfung von Claude Code	Betreuer rollt selbst aus (18:41)
02.11. 18:28	Download startet eine Stunde später als nötig	zu vorsichtig vorgebucht	harmlos; Buchführung genauer schätzen

Zeitleiste

Tabelle 20: Zeitleiste

Zeit (UTC)	Schritt	Commit
02.11. 08:00	Konzept geprüft, Tag 7 fehlt	
02.11. 08:14	Deploy-Key, Plan ins Repo	f151cfd
02.11. 08:20 bis 09:15	Phase 0: Vorlaufzeit, SMARD, Orte, Lückenprüfung	
02.11. 09:16	Phase 0 abgeschlossen	163d455
02.11. 18:28	Download gestartet, Archiv-Job geschrieben	552f040
02.11. 18:37	Protokollierung und Tagebuch, Download neu gestartet	109c6f3
02.11. 18:41	Archiv-Job auf der Web-Maschine live	b0783a5
02.11. abends	dieser Bericht	

16 Glossar

Tabelle 21: Glossar

Begriff	Bedeutung
Auslastung	Erzeugung geteilt durch installierte Leistung (0 bis 1)
Abregelung	Netzbetreiber oder Betreiber drosseln Anlagen, z. B. bei negativen Preisen
Bandbreite	Bereich zwischen p10 und p90
Gradient Boosting	viele kleine Bäume, jeder korrigiert die vorherigen
ICON	Wettermodell des DWD (D2, EU, global)
icon_seamless	Zusammenstellung von Open-Meteo aus D2, EU und global
Klimatologie	Messlatte aus Mittelwerten der Vorjahre
Lauf	eine Rechnung des Wettermodells mit festem Startzeitpunkt
MAE	mittlerer absoluter Fehler
MCP	Model Context Protocol, Schnittstelle zwischen KI-Assistenten und Werkzeugen
p10, p50, p90	10-, 50- und 90-%-Quantil der Vorhersage
Persistenz	Messlatte „wie gestern“
Pinball-Verlust	Fehlermaß, mit dem man Quantile lernt
Previous Runs API	Open-Meteo-Archiv mit Vorhersagen früherer Läufe
Prüfpunkt	Entscheidung, ob das Projekt weitergeht
Trefferquote	Anteil Stunden, in denen der echte Wert in der Bandbreite liegt
Vorlaufzeit	Zielstunde minus Startzeitpunkt des Laufs

Quellen: Bundesnetzagentur | SMARD.de (CC BY 4.0). Wetterdaten: Open-Meteo.com (CC BY 4.0), Modelle des DWD. Installierte Leistung: Energy-Charts, Fraunhofer ISE. Kartenumrisse: Natural Earth (gemeinfrei). Alle Diagramme ohne den Zusatz „Schema“ zeigen echte Werte, erzeugt mit docs/bericht/abbildungen.py am 02.11.2026.